SQL Portfolio

Querying databases for security analysis

Ugonna Ezenekwe

Google Cybersecurity Professional Certificate, Course 4: Tools of the Trade

Environment: MariaDB shell, organization database

Last updated: October 2026

This portfolio records the SQL labs and practice exercises I have completed. Each entry shows the security problem, the queries I wrote, what the results told me, and the mistakes I made and fixed along the way. New labs are added as I complete them, using the template at the end.

Contents

- Skills tracker
- Lab 1: Retrieve and sort data with SELECT, FROM and ORDER BY
- Practice exercise: Primary and foreign keys
- Template for the next lab

Skills tracker

SQL keywords I have used so far. I add a row each time I learn something new.

Keyword	What it does	Where I used it
`SELECT`	Chooses which columns to return	Lab 1, all tasks
`*`	Returns every column in the table	Lab 1, tasks 1 to 3
`FROM`	Names the table to query	Lab 1, all tasks
`;`	Ends the statement so it runs	Lab 1, all tasks
`ORDER BY`	Sorts the results by one or more columns	Lab 1, task 3; practice
`ASC / DESC`	Sorts A to Z (default) or Z to A	Practice exercise
`WHERE`	Filters rows that match a condition	Lab 1, extension
`PRIMARY KEY`	Column with a unique value for every row	Practice exercise
`FOREIGN KEY`	Column that must match a primary key in another table	Practice exercise

Lab 1: Retrieve and sort data with SELECT, FROM and ORDER BY

Scenario

My team needed to find out which employee devices must be updated, and whether any unusual login activity had happened. The data was in two tables in the organization database.

Table	Focus	Columns
`machines`	Hardware	device_id, operating_system, email_client, OS_patch_date, employee_id
`log_in_attempts`	User activity	event_id, username, login_date, login_time, country, ip_address, success

Task 1: Retrieve employee device data

First I returned everything in the machines table to see what data was available. The query returned 200 rows.

```
SELECT *
FROM machines;
```

Then I narrowed it to the email client on each device:

```
SELECT device_id, email_client
FROM machines;
```

Then the operating system and last patch date for each device:

```
SELECT device_id, operating_system, OS_patch_date
FROM machines;
```

Question	My answer
What email client is returned in the third row?	Email Client 2
What is the patch date of the first entry?	2021-09-01

Why it matters: An old OS_patch_date shows a device that may be missing security fixes. Those are the machines an attacker could exploit, so they are the first to update.

Task 2: Investigate login activity

The organisation operates in the United States, Canada and Mexico, so I checked where login attempts came from:

```
SELECT event_id, country
FROM log_in_attempts;
```

Then I checked who logged in and when, to spot logins outside working hours:

```
SELECT username, login_date, login_time
FROM log_in_attempts;
```

Finally I returned every column for a complete picture:

```
SELECT *
FROM log_in_attempts;
```

Question	My answer
Were any login attempts made from Australia?	No
What username is returned in the fifth row?	jrafael

Why it matters: A login from outside the countries the business operates in, or a login at 3am from someone who works days, can mean stolen credentials or an insider accessing data they should not.

Task 3: Order login attempts data

Scanning 200 unsorted rows is slow, so I sorted the login attempts by date:

```
SELECT *
FROM log_in_attempts
ORDER BY login_date;
```

Then I added login time as a second sort column. SQL sorts by login_date first, and only uses login_time to order rows that share the same date. The result is a full timeline.

```
SELECT *
FROM log_in_attempts
ORDER BY login_date, login_time;
```

Question	My answer
First record when sorted by login_date	ivelasco, 2022-05-08
First record when sorted by login_date, login_time	bsand, 00:19:11

Why it matters: A chronological timeline is the starting point of an incident investigation. It shows what happened in what order, such as several failed logins followed by a success.

Extension: Filtering with WHERE

Beyond the lab steps, I used WHERE to pull out only the logins from one country instead of scanning the whole table:

```
SELECT *
FROM log_in_attempts
WHERE country = 'USA';
```

This returned 41 of the 200 rows.

```
MariaDB [organization]> SELECT * FROM log_in_attempts WHERE country = 'USA';
+----------+----------+------------+------------+---------+-----------------+---------+
| event_id | username | login_date | login_time | country | ip_address      | success |
+----------+----------+------------+------------+---------+-----------------+---------+
|        3 | dkot     | 2022-05-09 | 06:47:41   | USA     | 192.168.151.162 |       1 |
|        4 | dkot     | 2022-05-08 | 02:00:39   | USA     | 192.168.178.71  |       0 |
|       12 | dkot     | 2022-05-08 | 09:11:34   | USA     | 192.168.100.158 |       1 |
|       13 | mrah     | 2022-05-11 | 09:29:34   | USA     | 192.168.246.135 |       1 |
|       15 | lyamamot | 2022-05-09 | 17:17:26   | USA     | 192.168.183.51  |       0 |
|       17 | pwashing | 2022-05-11 | 02:33:02   | USA     | 192.168.81.89   |       1 |
|       38 | sbaelish | 2022-05-09 | 14:40:01   | USA     | 192.168.60.42   |       1 |
|       48 | asundara | 2022-05-11 | 03:18:45   | USA     | 192.168.72.10   |       1 |
|       61 | dtanaka  | 2022-05-09 | 09:45:18   | USA     | 192.168.98.221  |       1 |
|       69 | wjaffrey | 2022-05-11 | 19:55:15   | USA     | 192.168.100.17  |       0 |
|       73 | zbernal  | 2022-05-10 | 17:46:45   | USA     | 192.168.80.46   |       0 |
|       83 | lrodriqu | 2022-05-08 | 08:10:23   | USA     | 192.168.67.69   |       1 |
|       86 | dtanaka  | 2022-05-10 | 10:22:20   | USA     | 192.168.197.135 |       1 |
|       89 | dkot     | 2022-05-12 | 10:52:00   | USA     | 192.168.128.75  |       1 |
|      107 | bisles   | 2022-05-12 | 20:25:57   | USA     | 192.168.116.187 |       0 |
|      110 | mabadi   | 2022-05-09 | 00:01:54   | USA     | 192.168.90.124  |       1 |
|      117 | bsand    | 2022-05-08 | 00:19:11   | USA     | 192.168.197.187 |       0 |
|      124 | asundara | 2022-05-12 | 10:51:21   | USA     | 192.168.136.29  |       1 |
|      129 | drosas   | 2022-05-12 | 15:39:40   | USA     | 192.168.152.200 |       0 |
|      130 | mrah     | 2022-05-11 | 02:54:21   | USA     | 192.168.102.147 |       0 |
|      133 | asundara | 2022-05-12 | 05:57:04   | USA     | 192.168.6.9     |       1 |
|      134 | iuduike  | 2022-05-09 | 06:46:40   | USA     | 192.168.22.115  |       1 |
|      141 | btang    | 2022-05-12 | 10:12:03   | USA     | 192.168.82.139  |       0 |
|      143 | jhill    | 2022-05-11 | 00:30:22   | USA     | 192.168.189.19  |       0 |
|      149 | jlansky  | 2022-05-11 | 01:07:11   | USA     | 192.168.238.42  |       0 |
|      151 | mabadi   | 2022-05-09 | 16:29:46   | USA     | 192.168.30.225  |       1 |
|      152 | mabadi   | 2022-05-12 | 10:24:43   | USA     | 192.168.96.244  |       0 |
|      155 | cgriffin | 2022-05-12 | 22:18:42   | USA     | 192.168.236.176 |       0 |
|      156 | btang    | 2022-05-11 | 17:08:51   | USA     | 192.168.243.95  |       0 |
|      159 | iuduike  | 2022-05-12 | 16:59:50   | USA     | 192.168.220.115 |       0 |
|      168 | jlansky  | 2022-05-08 | 13:25:42   | USA     | 192.168.210.94  |       1 |
|      170 | sbaelish | 2022-05-09 | 16:43:18   | USA     | 192.168.65.113  |       0 |
|      171 | drosas   | 2022-05-10 | 16:32:55   | USA     | 192.168.92.218  |       0 |
|      175 | jhill    | 2022-05-10 | 00:17:09   | USA     | 192.168.130.218 |       0 |
|      177 | wjaffrey | 2022-05-11 | 00:15:55   | USA     | 192.168.144.165 |       0 |
|      182 | lyamamot | 2022-05-10 | 06:01:31   | USA     | 192.168.106.52  |       0 |
|      185 | jsoto    | 2022-05-10 | 13:34:58   | USA     | 192.168.151.91  |       0 |
|      186 | bisles   | 2022-05-09 | 04:29:17   | USA     | 192.168.40.72   |       0 |
|      188 | jsoto    | 2022-05-11 | 00:39:09   | USA     | 192.168.21.88   |       0 |
|      190 | jsoto    | 2022-05-09 | 05:09:21   | USA     | 192.168.25.60   |       0 |
|      192 | bisles   | 2022-05-10 | 08:32:03   | USA     | 192.168.201.40  |       1 |
+----------+----------+------------+------------+---------+-----------------+---------+
41 rows in set (0.001 sec)
```

Output from the MariaDB shell: 41 rows in set

Next step: the same table may store a country in more than one form (for example a full name and an abbreviation), so an exact match can miss rows. Checking with `SELECT DISTINCT country` first, or using `LIKE`, would catch them.

Mistakes I made and how I fixed them

What I wrote	What went wrong	Fix
`FROM log_in_attempts` typed twice	ERROR 1064. I pressed Enter without a semicolon, so the -> prompt added my second line to the same query.	Write the query once and end it with ;. Type \c to cancel a half-written query.
`ORDER FROM country`	Not a valid keyword	`ORDER BY country`
`FROM employee_id`	FROM needs a table name, not a column name	`FROM employees`
`SET employee_id 2000`	Missing = sign, so it is a syntax error	`SET employee_id = 2000`

Practice exercise: Primary and foreign keys

Setup

I practised with two linked tables: employees and login_attempts.

- `employees.employee_id` is the primary key. Every employee has a unique ID.
- `login_attempts.event_id` is the primary key of the second table.
- `login_attempts.employee_id` is a foreign key. Every login attempt has to belong to an employee who exists.

```
CREATE TABLE login_attempts (
    event_id    INTEGER PRIMARY KEY,
    employee_id INTEGER NOT NULL,
    login_date  DATE,
    login_time  TIME,
    country     TEXT,
    success     INTEGER,
    FOREIGN KEY (employee_id) REFERENCES employees (employee_id)
);
```

Sorting on more than one column

Countries A to Z, then earliest login time first within each country:

```
SELECT country, employee_id, login_time
FROM login_attempts
ORDER BY country, login_time;
```

Departments A to Z, with usernames Z to A within each department:

```
SELECT employee_id, username, department
FROM employees
ORDER BY department ASC, username DESC;
```

employee_id	username	department
1006	fpatel	Finance
1001	asmith	Finance
1003	cjones	Human Resources
1004	dokafor	IT
1002	bkhan	IT
1005	ewilson	Sales

Testing the foreign key

I tried to point a login attempt at an employee who does not exist:

```
UPDATE login_attempts
SET employee_id = 2000
WHERE event_id = 2;
```

Expected result: the database refuses the change with `FOREIGN KEY constraint failed`, because there is no employee 2000.

> **Why it matters:** The foreign key stops the logs from containing activity for users who do not exist. In a real system, a log entry with no matching user would be a red flag.

Template for the next lab

Copy this section for each new lab and replace the prompts.

Lab title

Scenario

What security problem was I solving, and which tables did I use?

Queries

```
SELECT ...
FROM ...
WHERE ...
ORDER BY ...;
```

Findings

Question	My answer

Why it matters

How would a security analyst use this result?

Mistakes and fixes

What I wrote	What went wrong	Fix